

LIVRET D'EXCELLENCE TOUT-EN-UN

SCIENCES DE L'INFORMATIQUE (BAC TUNISIE)

Section : 4^{ème} Sciences de l'Informatique

CHAPITRE 2 : LA RÉCURSIVITÉ

Fiche 1 : Le Cours Complet & Démonstrations

Fiche 2 : Le Recueil de 40 Questions-Réponses

Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo

Plateforme Académique d'Excellence :

mathbacai.com

Conforme aux exigences du programme officiel du Ministère de l'Éducation Tunisien

Contents

1	Fiche 1 : Le Cours Complet	2
1.1	I. Introduction & Fondements de la Récursivité	2
1.1.1	1. Définition Conceptuelle	2
1.1.2	2. Fonctionnement de la Pile d'Exécution	2
1.1.3	3. Types de Récursivité	2
1.1.4	4. Comparaison : Récursif vs Itératif	3
1.2	II. Conception d'un Algorithme Récursif	3
1.3	III. Les Applications Majeures du Programme (Algorithmes & Pascal)	4
1.3.1	1. Le Calcul de la Factorielle d'un Entier	4
1.3.2	2. La Somme des N Premiers Entiers	4
1.3.3	3. Le Calcul de la Puissance Récursive (a^n)	5
1.3.4	4. Le Test de Palindrome d'une Chaîne	5
1.3.5	5. La Recherche Dichotomique Récursive	6
2	Fiche 2 : Le Recueil de Questions-Réponses	8
3	Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo	13
3.1	I. Quiz d'Évaluation de l'Excellence	13
3.2	II. Le Grand Tableau Récapitulatif des Applications	16
3.3	III. Les 5 Règles d'Or de l'Excellence Récursive	16

Chapter 1

Fiche 1 : Le Cours Complet

1.1 I. Introduction & Fondements de la Récursivité

1.1.1 1. Définition Conceptuelle

En informatique et en algorithmique, la **récursivité** est une méthode de résolution de problèmes qui consiste à définir un sous-programme (fonction ou procédure) de telle sorte qu'il fasse référence à **lui-même** dans son propre corps.

Cette approche s'oppose au modèle **itératif** (qui utilise des structures de contrôle répétitives de type boucles Pour, **Tant que**, ou Répéter). Elle s'inspire directement du principe mathématique de **récurrence**, où un terme d'ordre n est calculé à partir de termes d'ordre inférieur.

1.1.2 2. Fonctionnement de la Pile d'Exécution

Chaque appel d'un sous-programme en informatique nécessite l'utilisation d'une zone de mémoire dédiée appelée **Pile d'Exécution** (*Execution Stack*). Dans le cadre de la récursivité, ce mécanisme est crucial mais comporte des limites techniques strictes :

- **L'empilement (Push)** : À chaque appel récursif, un nouvel **Enregistrement d'Activation** (ou *Stack Frame*) est empilé. Cet enregistrement contient :
 1. L'adresse de retour (pour savoir où reprendre l'exécution dans le programme appelant).
 2. Les valeurs des paramètres formels transmis.
 3. Les variables locales propres à cet appel.
- **Le point de blocage temporaire** : Tant que la *condition d'arrêt* n'est pas atteinte, aucun appel ne se termine. Les enregistrements s'accumulent dans la pile, maintenant les calculs en suspens.
- **Le dépilement (Pop)** : Dès que la condition d'arrêt est validée, le sous-programme renvoie une valeur concrète. La pile commence à se vider : les calculs intermédiaires en suspens sont résolus de manière ascendante (du dernier appel vers le premier).
- **Le risque de Débordement de Pile (*Stack Overflow*)** : Si le nombre d'appels récursifs est trop important ou si la condition d'arrêt est absente ou erronée, la pile sature. En Pascal, cela déclenche une erreur d'exécution fatale (*Stack overflow error*).

1.1.3 3. Types de Récursivité

On distingue plusieurs formes de récursivité selon la structure des appels :

- **Récursivité Directe** : Le sous-programme P s'appelle directement lui-même ($P \rightarrow P$).
- **Récursivité Indirecte (ou mutuelle)** : Le sous-programme P appelle un sous-programme Q , qui à son tour appelle P ($P \rightarrow Q \rightarrow P$).

- **Récursivité Simple (ou linéaire)** : Chaque cas général ne contient qu'un seul appel récursif (ex: Factorielle, Somme).
- **Récursivité Multiple** : Un cas général engendre plusieurs appels récursifs (ex: Suite de Fibonacci qui nécessite deux appels, ou les Tours de Hanoï).
- **Récursivité Terminale (Tail Recursion)** : L'appel récursif est la toute dernière opération effectuée par le sous-programme. Aucune opération de calcul ne reste en suspens après l'appel. Elle est très facile à dérécurifier de façon automatique par les compilateurs.

1.1.4 4. Comparaison : Récursif vs Itératif

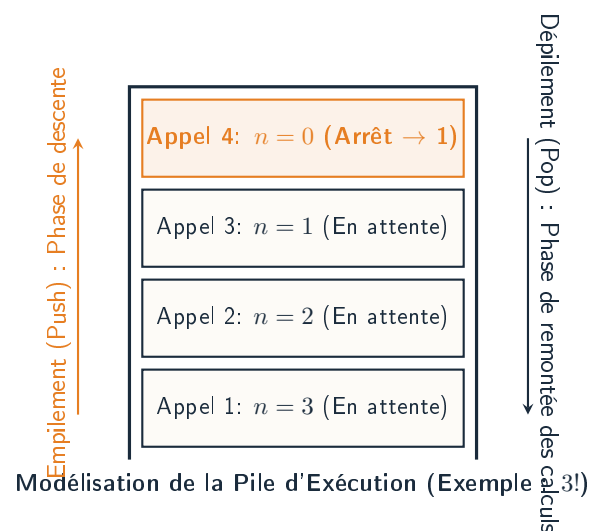
Critère	Approche Récursive	Approche Itérative
Écriture	Plus simple, élégante et proche de l'esprit mathématique.	Parfois complexe pour les structures d
Mémoire	Gourmande (utilisation intensive de la pile d'exécution).	Économe (allocation fixe de mémoire)
Performance	Temps système plus long (dû au mécanisme d'appel/retour).	Généralement plus rapide et directe.
Sécurité	Risque élevé de plantage (<i>Stack Overflow</i>).	Risque de boucles infinies (consomme

Table 1.1: Comparaison technique des deux paradigmes de pensée

1.2 II. Conception d'un Algorithme Récursif

La conception d'un module récursif repose impérativement sur une analyse par cas structurée autour de deux éléments fondamentaux :

1. **Le Cas Particulier (Base de récurrence ou Condition d'arrêt)** : C'est le point d'appui du traitement. Il définit la condition sous laquelle le sous-programme cesse de s'appeler lui-même et renvoie immédiatement une valeur fixe (ou effectue une action terminale). Sans cela, l'algorithme tournerait indéfiniment.
2. **Le Cas Général (Appel récursif)** : C'est l'étape où le problème est décomposé en un sous-problème de taille plus réduite. L'argument transmis lors de l'appel récursif **doit impérativement converger** vers la condition d'arrêt.



1.3 III. Les Applications Majeures du Programme (Algorithmes & Pascal)

1.3.1 1. Le Calcul de la Factorielle d'un Entier

La factorielle d'un entier naturel n (notée $n!$) est définie par :

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \times (n-1)! & \text{si } n > 0 \end{cases}$$

Algorithme & Traduction Pascal : Factorielle Récursive

En Algorithmique :

```
0) Fonction Factorielle (n : Entier) : Entier Long
1) Si (n = 0) Alors
    Factorielle <- 1
  Sinon
    Factorielle <- n * Factorielle(n - 1)
  Fin Si
2) Fin Factorielle
```

En Langage Pascal :

```
Function Factorielle (n : LongInt) : LongInt;
Begin
  If (n = 0) Then
    Factorielle := 1
  Else
    Factorielle := n * Factorielle(n - 1);
End;
```

1.3.2 2. La Somme des N Premiers Entiers

Calcul de la somme $S(n) = 1 + 2 + 3 + \dots + n$.

$$S(n) = \begin{cases} 0 & \text{si } n = 0 \\ n + S(n-1) & \text{si } n > 0 \end{cases}$$

Algorithme & Traduction Pascal : Somme Récursive

En Algorithmique :

```
0) Fonction Somme (n : Entier) : Entier
1) Si (n = 0) Alors
    Somme <- 0
  Sinon
    Somme <- n + Somme(n - 1)
  Fin Si
2) Fin Somme
```

En Langage Pascal :

```
Function Somme (n : Integer) : Integer;
```

```

Begin
  If (n = 0) Then
    Somme := 0
  Else
    Somme := n + Somme(n - 1);
End;

```

1.3.3 3. Le Calcul de la Puissance Récursive (a^n)

Pour calculer a^n (avec $a \in \mathbb{R}$ et $n \in \mathbb{Z}$), nous modélisons les cas particuliers et généraux :

$$\text{Puissance}(a, n) = \begin{cases} 1 & \text{si } n = 0 \\ 1/\text{Puissance}(a, -n) & \text{si } n < 0 \\ a \times \text{Puissance}(a, n - 1) & \text{si } n > 0 \end{cases}$$

Algorithme & Traduction Pascal : Puissance Récursive

En Algorithmique :

```

0) Fonction Puissance (a : Réel ; n : Entier) : Réel
1) Si (n = 0) Alors
    Puissance <- 1
  Sinon Si (n < 0) Alors
    Puissance <- 1 / Puissance(a, -n)
  Sinon
    Puissance <- a * Puissance(a, n - 1)
  Fin Si
2) Fin Puissance

```

En Langage Pascal :

```

Function Puissance (a : Real; n : Integer) : Real;
Begin
  If (n = 0) Then
    Puissance := 1
  Else If (n < 0) Then
    Puissance := 1 / Puissance(a, -n)
  Else
    Puissance := a * Puissance(a, n - 1);
End;

```

1.3.4 4. Le Test de Palindrome d'une Chaîne

Une chaîne de caractères est un palindrome si elle se lit de la même façon dans les deux sens (ex: "radar", "rotor").

- **Cas Particuliers** : Si la longueur de la chaîne est inférieure à 2, elle est automatiquement un palindrome.
- **Cas Général** : Si le premier et le dernier caractère sont identiques, le résultat dépend de la même vérification sur la sous-chaîne privée de ses extrémités. Sinon, la chaîne n'est pas un palindrome.

Algorithme & Traduction Pascal : Palindrome Récursif**En Algorithmique :**

```

0) Fonction Palindrome (ch : Chaîne) : Booléen
1) Si Long(ch) < 2 Alors
    Palindrome <- Vrai
  Sinon Si ch[1] = ch[Long(ch)] Alors
    Palindrome <- Palindrome(Sous_chaîne(ch, 2, Long(ch) - 2))
  Sinon
    Palindrome <- Faux
  Fin Si
2) Fin Palindrome

```

En Langage Pascal :

```

Function Palindrome (ch : String) : Boolean;
Begin
  If Length(ch) < 2 Then
    Palindrome := True
  Else If (ch[1] = ch[Length(ch)]) Then
    Palindrome := Palindrome(Copy(ch, 2, Length(ch) - 2))
  Else
    Palindrome := False;
End;

```

1.3.5 5. La Recherche Dichotomique Récursive

La recherche dichotomique s'applique exclusivement sur un tableau trié. Elle consiste à diviser par 2 l'espace de recherche à chaque étape.

Algorithme & Traduction Pascal : Recherche Dichotomique Récursive**En Algorithmique :**

```

0) Fonction Dichotomique (deb, fin, m : Entier; T : Tab) : Booléen
1) milieu <- (deb + fin) Div 2
  Si (m = T[milieu]) Alors
    Dichotomique <- Vrai
  Sinon Si (m < T[milieu]) Et (deb < milieu) Alors
    Dichotomique <- Dichotomique(deb, milieu - 1, m, T)
  Sinon Si (m > T[milieu]) Et (fin > milieu) Alors
    Dichotomique <- Dichotomique(milieu + 1, fin, m, T)
  Sinon
    Dichotomique <- Faux
  Fin Si
2) Fin Dichotomique

```

En Langage Pascal :

```

Type Tab = Array[1..100] Of Integer;

Function Dichotomique (deb, fin, m : Integer; T : Tab) : Boolean;

```

```
Var milieu : Integer;  
Begin  
  milieu := (deb + fin) Div 2;  
  If (m = T[milieu]) Then  
    Dichotomique := True  
  Else If (m < T[milieu]) And (deb < milieu) Then  
    Dichotomique := Dichotomique(deb, milieu - 1, m, T)  
  Else If (m > T[milieu]) And (fin > milieu) Then  
    Dichotomique := Dichotomique(milieu + 1, fin, m, T)  
  Else  
    Dichotomique := False;  
End;
```

Chapter 2

Fiche 2 : Le Recueil de Questions-Réponses

Voici 40 questions d'évaluation active avec leurs réponses d'excellence, conçues pour couvrir tous les points sensibles du programme de la classe de 4^{ème} Sciences de l'Informatique.

Question 1

L'approche itérative utilise des boucles de répétition (Pour, Tant que, Répéter) avec des structures de contrôle explicites, tandis que l'approche récursive résout le problème en demandant au sous-programme de s'appeler lui-même avec un paramètre de taille réduite jusqu'à l'atteinte d'un cas élémentaire.

Question 2

Le cas d'arrêt est la condition (ou l'ensemble de conditions) qui met fin à l'enchaînement des appels récursifs et renvoie immédiatement une valeur sans faire de nouvel appel à lui-même. C'est l'élément indispensable pour éviter une récursion infinie.

Question 3

Le programme s'appelle lui-même indéfiniment en empilant de manière ininterrompue des enregistrements d'activation dans la pile d'exécution, jusqu'à saturer totalement la mémoire allouée à la pile, provoquant un plantage du système appelé « Débordement de pile » (*Stack Overflow*).

Question 4

C'est un bloc mémoire temporaire créé et empilé à chaque appel de sous-programme. Il contient l'adresse de retour (pour continuer l'exécution après l'appel), les valeurs des paramètres d'entrée et toutes les variables locales déclarées à l'intérieur du module.

Question 5

Le système d'exploitation utilise une structure de type **Pile** (LIFO - *Last In, First Out* : Dernier Entré, Premier Sorti). Les appels s'empilent lors de la descente et se dépilent de manière inverse lors de la remontée des calculs.

Question 6

La récursivité est dite directe lorsqu'un module comporte, dans son propre corps de code, au moins un appel direct à lui-même (Exemple : la fonction Factorielle s'appelant elle-même).

Question 7

La récursivité directe est un appel de soi par soi ($A \rightarrow P$). La récursivité croisée implique plusieurs modules qui s'appellent mutuellement (Exemple : A appelle B , qui à son tour appelle A).

Question 8

C'est le processus technique qui consiste à transformer un algorithme récursif en un algorithme itératif équivalent. Cette étape est souvent nécessaire pour optimiser la mémoire vive et éviter les limites physiques de la pile d'exécution.

Question 9

Oui, théoriquement, tout algorithme écrit de manière récursive possède une version itérative équivalente, bien que l'écriture itérative puisse être beaucoup plus complexe et nécessiter la gestion manuelle d'une structure de pile de données.

Question 10

C'est un sous-programme récursif où l'appel récursif est la toute dernière instruction exécutée. Il n'y a aucune opération en suspens (aucune multiplication ou addition) après l'appel, ce qui permet aux compilateurs d'optimiser l'espace de la pile.

Question 11

Parce que chaque appel récursif implique un surcoût système (gestion de la pile d'exécution, copie des arguments, changement de contexte processeur et sauvegarde des variables locales).

Question 12

Le principal avantage est la clarté et la concision du code. Elle permet d'exprimer des solutions complexes d'une manière naturelle, lisible et extrêmement proche de leur définition mathématique d'origine.

Question 13

Il faut démontrer rigoureusement que les paramètres transmis lors de chaque appel récursif évoluent et se rapprochent de manière stricte de la condition qui valide le cas d'arrêt.

Question 14

C'est la valeur $n = 0$. À cette étape, la fonction renvoie directement 1 sans effectuer de nouvel appel récursif.

Question 15

La condition d'arrêt ($n = 0$) ne serait jamais vérifiée car à chaque appel récursif, la valeur décroît ($n - 1, n - 2, \dots$). La récursion se poursuivrait indéfiniment jusqu'au débordement de la pile d'exécution.

Question 16

Elle est de $O(n)$ car chaque appel nécessite l'allocation d'un espace mémoire (Stack Frame) dans la pile d'exécution. Pour un entier n , il y aura $n + 1$ appels simultanés en mémoire.

Question 17

C'est un traitement récurrent dans lequel le calcul d'un résultat courant nécessite la connaissance des p résultats précédemment obtenus (Exemple : Fibonacci est d'ordre 2 car $U_n = U_{n-1} + U_{n-2}$).

Question 18

L'appel $3!$ attend le retour de $2!$, qui attend $1!$, qui attend $0!$. Une fois $0! = 1$ renvoyé, le système remonte : $1! = 1 \times 1 = 1$, puis $2! = 2 \times 1 = 2$, et enfin $3! = 3 \times 2 = 6$.

Question 19

La variable globale est allouée dans la mémoire globale (zone de données statiques) du programme. Elle n'est pas copiée dans la pile d'exécution à chaque appel, ce qui signifie que sa modification par un appel affecte sa valeur pour tous les autres appels en attente.

Question 20

Elle est indispensable pour le parcours et la manipulation de structures de données intrinsèquement récursives (comme les arbres binaires, les graphes ou le tri rapide / tri fusion).

Question 21

La relation est : $F(n) = F(n-1) + F(n-2)$ pour $n > 2$, avec comme conditions d'arrêt : $F(1) = 1$ et $F(2) = 1$.

Question 22

Car elle effectue de multiples calculs redondants. Pour calculer $F(5)$, elle calcule $F(4)$ et $F(3)$, mais $F(4)$ va lui-même recalculer $F(3)$. La complexité temporelle est exponentielle $O(2^n)$.

Question 23

Ce procédé s'appelle la **mémorisation** (ou *Memoization*), une technique fondamentale de la programmation dynamique.

Question 24

$PGCD(a, b) = a$ si $a = b$. Sinon, si $a > b$, $PGCD(a, b) = PGCD(a - b, b)$. Sinon, $PGCD(a, b) = PGCD(a, b - a)$.

Question 25

Si $p = 0$ ou $p = n$, $C_n^p = 1$. Sinon, $C_n^p = C_{n-1}^p + C_{n-1}^{p-1}$ (Règle du triangle de Pascal).

Question 26

Si l'exposant n est inférieur à 0, $a^n = 1/Puissance(a, -n)$. L'appel récursif convertit l'exposant en positif pour être traité de manière convergente.

Question 27

Le Triangle de Pascal est un traitement récurrent d'ordre 2 car chaque cellule dépend de deux cellules situées sur la ligne supérieure ($MAT[l, c] = MAT[l - 1, c] + MAT[l - 1, c - 1]$).

Question 28

Il y a deux conditions d'arrêt possibles : soit l'élément recherché est trouvé à l'indice médian, soit l'espace de recherche devient invalide ($deb > fin$), indiquant que l'élément n'existe pas.

Question 29

La taille de l'espace de recherche est divisée par 2 à chaque appel récursif. Si l'élément recherché est plus petit que la valeur médiane, la recherche continue entre *deb* et *milieu* - 1, sinon entre *milieu* + 1 et *fin*.

Question 30

La fonction `Copy(ch, 2, Length(ch) - 2)` permet d'extraire la sous-chaîne de caractères en excluant la première et la dernière lettre de la chaîne actuelle, pour l'envoyer au prochain appel récursif.

Question 31

Lorsque la chaîne de caractères transmise a une longueur inférieure à 2 (c'est-à-dire une longueur de 0 ou de 1).

Question 32

Pour éviter de dupliquer l'intégralité du tableau en mémoire dans la pile d'exécution à chaque appel récursif. En utilisant le passage par référence (VAR), seul un pointeur d'adresse est empilé.

Question 33

Oui, tout à fait. Une procédure peut s'appeler elle-même récursivement. L'absence de valeur de retour directe est compensée par la modification de paramètres passés par référence (VAR).

Question 34

Il ne la détecte pas à la compilation. La compilation s'effectue avec succès, mais le plantage (*Stack Overflow*) se produit uniquement lors de l'exécution (*Runtime*) du programme.

Question 35

La taille de la pile dépend de la configuration du compilateur (souvent 16 Ko ou 64 Ko par défaut dans l'environnement Turbo Pascal). Les compilateurs modernes comme Free Pascal ou Lazarus permettent de la configurer de manière dynamique.

Question 36

Elle est récursive terminale car l'appel récursif est la toute dernière opération : aucune autre manipulation mathématique n'est effectuée sur le résultat renvoyé par l'appel.

Question 37

Il déplace récursivement $n - 1$ disques de la tour de départ vers la tour intermédiaire, déplace le plus grand disque de la tour de départ vers la tour cible, puis déplace de nouveau les $n - 1$ disques de la tour intermédiaire vers la tour cible.

Question 38

Le nombre total de déplacements physiques est de $2^n - 1$. C'est un algorithme de complexité exponentielle $O(2^n)$.

Question 39

C'est une technique algorithmique récursive de recherche systématique qui consiste à avancer pas à pas vers une solution et à rebrousser chemin (revenir en arrière sur les décisions) dès que l'on se heurte à un blocage.

Question 40

C'est le problème des huit dames (placer 8 dames sur un échiquier de 8×8 cases sans qu'elles ne se menacent mutuellement).

Chapter 3

Fiche 3 : Le Quiz d'Évaluation & Fiches Mémo

3.1 I. Quiz d'Évaluation de l'Excellence

Ce quiz de 10 questions représentatives du Baccalauréat tunisien permet de valider de manière définitive vos acquis. Les réponses sont suivies d'analyses théoriques détaillées.

Question 1 : Vrai ou Faux ?

Proposition : Un sous-programme récursif consomme moins de mémoire RAM que sa version itérative équivalente.

Réponse : FAUX.

Explication technique : La version récursive nécessite à chaque appel la sauvegarde de l'état du programme et l'allocation d'un espace temporaire dans la pile d'exécution. À l'inverse, l'itérative conserve un espace constant.

Question 2 : QCM

Énoncé : Quel est l'ordre de récurrence de la suite de Fibonacci définie par $U_n = U_{n-1} + U_{n-2}$?

- a) Ordre 1
- b) Ordre 2
- c) Ordre n

Réponse : b) Ordre 2.

Justification : Chaque terme U_n nécessite l'utilisation des deux états qui le précèdent immédiatement en mémoire (les rangs $n - 1$ et $n - 2$).

Question 3 : Vrai ou Faux ?

Proposition : Un algorithme récursif terminal est plus facile à convertir en boucle de manière itérative qu'un algorithme non-terminal.

Réponse : VRAI.

Explication technique : La récursivité terminale ne requiert aucun traitement après l'appel. Elle peut être convertie en une simple itération sans avoir besoin d'allouer une structure de pile

auxiliaire.

Question 4 : QCM

Énoncé : Dans une fonction récursive qui calcule a^n , si l'exposant n est strictement négatif, comment converge-t-on ?

- a) En arrêtant le programme avec une erreur.
- b) En renvoyant $1/Puissance(a, -n)$.
- c) En soustrayant 1 à n à chaque étape.

Réponse : b) En renvoyant $1/Puissance(a, -n)$.

Justification : Cette opération transforme l'exposant négatif en sa valeur absolue positive, permettant ainsi une récursion descendante classique et contrôlée vers le cas d'arrêt $n = 0$.

Question 5 : Vrai ou Faux ?

Proposition : Le test de dépassement de la pile (Stack Overflow) est un contrôle effectué automatiquement par le compilateur lors de la compilation.

Réponse : FAUX.

Explication technique : Le compilateur ne peut pas prévoir à l'avance le nombre d'appels dynamiques. Ce dépassement survient exclusivement au cours de l'exécution (*Runtime*) du programme.

Question 6 : QCM

Énoncé : Quel est le nombre total de déplacements de disques nécessaires pour résoudre le problème des Tours de Hanoï pour un jeu à 4 disques ?

- a) 7
- b) 15
- c) 16

Réponse : b) 15.

Justification : Le nombre de déplacements physiques obéit à la relation mathématique $2^n - 1$. Pour $n = 4$, nous obtenons $2^4 - 1 = 16 - 1 = 15$.

Question 7 : Vrai ou Faux ?

Proposition : Le passage de paramètres par référence (mot-clé VAR en Pascal) permet d'optimiser la mémoire de la pile d'exécution lors d'appels récursifs.

Réponse : VRAI.

Explication technique : Au lieu d'empiler une copie complète des données (comme un grand tableau d'enregistrements), le système transmet uniquement l'adresse mémoire du tableau d'origine, économisant ainsi de l'espace sur la pile d'exécution.

Question 8 : QCM

Énoncé : Quelle est la condition d'arrêt par défaut de la fonction récursive de recherche dichotomique ?

- a) L'élément recherché est trouvé ou l'intervalle devient invalide ($\text{deb} > \text{fin}$).
- b) Le milieu du tableau est atteint.
- c) Le tableau est entièrement parcouru du début à la fin.

Réponse : a) L'élément recherché est trouvé ou l'intervalle devient invalide ($\text{deb} > \text{fin}$).

Justification : Si l'élément est à l'indice médian, on s'arrête. Si l'élément n'existe pas, l'indice deb dépasse fin , ce qui invalide l'espace de recherche.

Question 9 : Vrai ou Faux ?

Proposition : Un algorithme de tri comme le Tri par Sélection peut être formulé récursivement sans nécessiter l'utilisation d'une boucle imbriquée.

Réponse : VRAI.

Explication technique : L'itération externe du tri par sélection peut être remplacée par des appels récursifs sur le reste du tableau non trié.

Question 10 : QCM

Énoncé : Dans une fonction récursive de calcul de Somme des N premiers entiers, que retourne la fonction si elle reçoit $n = 0$?

- a) 1
- b) 0
- c) Elle génère une erreur.

Réponse : b) 0.

Justification : C'est la valeur élémentaire de base d'une somme d'ensemble vide. La somme d'aucun entier est égale à 0.

3.2 II. Le Grand Tableau Récapitulatif des Applications

Tableau de Mémorisation Universelle			
Application	Condition d'Arrêt	Formule Générale / Appel Récursif	Complexité
Factorielle	$n = 0 \rightarrow 1$	$n \times \text{Factorielle}(n - 1)$	$O(n)$
Somme	$n = 0 \rightarrow 0$	$n + \text{Somme}(n - 1)$	$O(n)$
Puissance	$n = 0 \rightarrow 1$	$a \times \text{Puissance}(a, n - 1)$ (si $n > 0$)	$O(n)$
Palindrome	$\text{Long}(ch) < 2 \rightarrow \text{Vrai}$	$\text{Palindrome}(\text{Copy}(ch, 2, \text{Long} - 2))$ (si $ch[1] = ch[L]$)	$O(n)$
Dichotomie	$m = T[mid] \rightarrow \text{Vrai}$	$\text{Dichotomique}(deb, mid - 1, m, T)$ (si $m < T[mid]$)	$O(\log n)$
Hanoï	$n = 0 \rightarrow \text{Rien}$	$2 \times \text{Hanoi}(n - 1) + \text{Déplacement principal}$	$O(2^n)$

Table 3.1: Synthèse absolue des algorithmes récursifs officiels

3.3 III. Les 5 Règles d'Or de l'Excellence Récursive

1. **La Condition d'Arrêt d'abord** : Ne rédigez jamais le corps d'un module sans placer explicitement la condition d'arrêt au tout début du code.
2. **La Garantie de Convergence** : Validez toujours que le paramètre transmis lors de l'appel récursif décroît ou évolue d'une façon à atteindre la condition d'arrêt dans tous les cas d'utilisation possibles.
3. **L'Optimisation Mémoire** : Pour les tableaux et objets lourds, utilisez impérativement le passage par adresse (mot-clé VAR) pour économiser l'espace précieux de la pile.
4. **La Détection du Risque** : Évitez la récursivité multiple (comme Fibonacci naïf) pour des volumes importants de données en raison de sa complexité exponentielle, et préférez une version itérative ou dynamique.
5. **L'Échappement des Pièges Pascal** : Attention aux indices de chaînes ! En Pascal, la notation d'extraction Copy nécessite un contrôle rigoureux de la longueur de la chaîne pour éviter les erreurs d'exécution d'accès à la mémoire.